

Scale and Synthesis: Fleet Execution and Derived Plans

Sean Chatman
with the Praxis working system

Abstract

This paper consolidates two sources in the Chatman Equation thesis series: *04_projection_and_scale*, which instantiates the keystone's Comprehension–Verification Gap theorem at fleet scale and derives a decidable-oracle discipline, a branchless byte-level admission construction, and a receipting invariant bounding claim magnitude; and *synthesis_thesis*, which derives – rather than hand-authors – the supervision topology and failure geometry that keep a fleet of agents running under fault, and reports the resulting 10,000-member cell's measured fault-injection numbers. Every paragraph below carries a machine-checkable evidentiary register, EVIDENCE-BOUND or SPECULATIVE; the planetary-scale economics chapter and one flagged proposition in the Fable-harness chapter are the only speculative material, and neither is mixed silently with the evidence-bound chapters around it.

Contents

1	Introduction	3
1.1	Thesis statement	3
1.2	What is cited from the keystone, not re-derived	3
1.3	A borrowed discipline, briefly (speculative)	4
1.4	Structure of this document	4
2	The Comprehension–Verification Gap, Instantiated at Scale	6
2.1	Measured and estimated quantities	6
2.2	The measured ratio, not an estimate	6
2.3	Boundary cost is invariant as the interior grows	6
2.4	Per-message wall time: an estimate, not a measurement	7
3	Manufactured Trust: Decidable Oracles	8
3.1	Why admission needs a decidable oracle	8
3.2	Differential agreement between independent implementations	8
3.3	Admission-boundary fuzzing	9
3.4	The Mutant Kill Theorem	9
4	The Agent8 Byte and Branchless Fleet Admission	10
4.1	The status byte as an OK-polarity projection	10
4.2	Bit-plane layouts and the branchless admission word	11
4.3	The denial-polarity dual	11
4.4	Borrow-safe SWAR zero-lane detection	11
4.5	Popcount admission in one instruction	11
4.6	Memory footprint: an exact identity	12
4.7	Sweep time: an order-of-magnitude estimate	12
5	Planetary Economics of Admission (Speculative)	13
5.1	Demand	13
5.2	Comprehension-based supply ceiling	13
5.3	Bit-parallel supply	13
5.4	The affordability separation theorem	13
6	The Antibody Clause: Claim Magnitude Bounded by Receipting	15
6.1	Claims and receipt witnesses	15
6.2	The Magnitude Theorem	15
6.3	Reflexive application: this paper’s own overclaim set	15

7	Derived Supervision Topology	17
7.1	Stage depth from the data-dependency DAG	17
7.2	Restart strategy earned from the dependent set	17
7.3	The sealed derive constructor and topology_hash	17
7.4	Refusal: no OneForAll	18
7.5	Refusal: no SimpleOneForOne, no clamped intensity	18
8	Derived Failure Geometry	19
8.1	The eight-member FailureClass enum	19
8.2	Branches and first-match classification	19
8.3	Completeness by construction: the gap outcome	19
8.4	Fragile preconditions	20
8.5	Total accounting	20
9	Supervised Execution: Cell Measurements	21
9.1	Recovery is invisible in the receipt chain	21
9.2	The 10,000-member cell	21
9.3	What flat throughput means	21
9.4	The deferred novelty-curve measurement	22
10	Fleet Application Tooling: Retrofit and Reconciliation	23
10.1	AST gates and the isolated worktree applier	24
10.2	Git-worktree isolation end to end	24
10.3	Autonomic reconciliation: residual vectors and gap reports	24
11	Fable Harness: Spec-Driven LLM Evaluation	26
11.1	Harness mechanics	26
11.2	Receipting fable runs	26
11.3	Speculative: decidability and differential-oracle claims	27
12	Hierarchical Merkle Cell Rollups and Write-Ahead Logs	28
12.1	Cell rollup	28
12.2	Write-ahead log frames	28
13	Scope Discipline: The Refusal Register	29
13.1	No path dependency on knhk	29
13.2	No measured CPU ticks	29
13.3	v1 scope, named	29
14	Conclusion	31

Chapter 1

Introduction

1.1 Thesis statement

This paper consolidates two sources that instantiate the same admit/manufacture/receipt pattern – $\mathcal{A} = \mu(\mathcal{O}^*)$, the Chatman equation of the keystone paper [1] – at fleet scale, on opposite sides of one boundary. `docs/thesis/04_projection_and_scale.tex` argues that admitting and re-admitting a fleet of computational agents cannot be a matter of comprehension: it must be mechanized into a decidable oracle, a branchless byte-level construction, and a receipting invariant that bounds how large a claim is allowed to be. Chapters 2–6 instantiate that source. `docs/thesis/synthesis_thesis.tex` argues the complementary half: the supervision structure that keeps such a fleet running – which nodes restart together, which failures are named in advance, which recovery claims are actually measured rather than assumed – can itself be derived from the plan, not hand-authored. Chapters 7–13 instantiate that source. Together they cover one lifecycle: admit a fleet, then keep it running when parts of it fail.

Every paragraph below carries a machine-checkable `doc:register` value, EVIDENCE-BOUND or SPECULATIVE, per `packs/doc-pack/ontology.ttl`. An evidence-bound paragraph names the artifact its claim traces to: a corpus label, a source file, or a receipts-file line. A speculative paragraph is motivated by evidence-bound results but is not itself derived from or checked against them, and is never left unmarked next to an evidence-bound claim in the same section. Chapter 5 and the second section of Chapter 11 are wholly speculative and are titled as such. Three further sections carry individually marked speculative paragraphs inside otherwise evidence-bound chapters – the borrowed-discipline-lineage section of Chapter 1 (titled “(speculative)” in its own heading), the wall-time estimate of Section 2.4 in Chapter 2, and the two order-of-magnitude estimates of the sweep-time section in Chapter 4 – each set off with an explicit **Speculative.** marker and never left ambiguous next to the evidence-bound arithmetic it sits beside.

1.2 What is cited from the keystone, not re-derived

Two results recur across this pair of sources but belong, proved once, to earlier papers in the series, and are cited rather than re-derived here. First, Rice’s theorem specialized to the observation space – for any non-trivial semantic property P of what an observation denotes, $\{o \in \mathcal{O} : P(o)\}$ is undecidable – is proved in `docs/thesis/00_foundations.tex` (label `thm:rice`) and restated at `docs/thesis/projection_thesis.tex` (same label); it is the reason admission must be a mechanical retraction and not a semantic decision, and Chapter 3 below only draws the operational consequence for oracle construction. Second, the Comprehension–Verification Gap theorem – boundary verification

cost is $O(\kappa)$ while comprehension cost grows without bound with the interior – is proved in `docs/thesis/projection_thesis.tex` (label `thm:gap`, chapter “The Two Cost Regimes and the Comprehension–Verification Gap”) [1]. Chapter 2 below instantiates that theorem with one measured fleet-scale number; it does not reprove it.

1.3 A borrowed discipline, briefly (speculative)

Speculative. `docs/thesis/synthesis_thesis.tex` motivates its derived-supervision design by analogy to three prior disciplines, kernel-admitted as `lineage` statements in `tools/paper-factory/lean-pilot/for` and `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl` (both verified, kernel 4.31.0), but a kernel admission of a lineage statement certifies only that the analogy was formalized as declared, not that the analogy is load-bearing for any claim in this paper – these are historical remarks, condensed to one paragraph rather than three separate chapters, and are not cited again below.

Lineage 1.1 (Erlang/OTP). Erlang/OTP made three moves that survive every re-examination: processes are isolated so failure cannot spread by memory; supervisors – not the failing code – own the recovery decision; and restart strategies plus a restart intensity turn ‘let it crash’ into a bounded, structured discipline. What OTP does not do is derive the tree: a human writes the supervision hierarchy, and the crash space lives in the programmer’s head.

Lineage 1.2 (Instrument design). The LHC does not record every collision; its trigger system encodes, in advance, the signatures of events worth keeping. The EHT did not photograph a black hole; it reconstructed one from lawful projections gathered by an instrument designed around a predicted signal. The discipline common to both: enumerate the event geometry before the event, instrument for exactly those signatures, and treat anything outside the geometry as a first-class finding – never as noise to be discarded silently.

Lineage 1.3 (The eight-bounded actor lineage). The 8-bounded actor lineage ($\text{CNS} \rightarrow \text{BitActor} \rightarrow \text{ByteActor} \rightarrow \text{knhk}$): CNS fixed the doctrine 8T/8H/8M; ByteActor V3 productized it with 64-byte crystal envelopes and bounded SPSC rings; knhk carried the doctrine into Rust with a branchless TickBudget, a ChatmanBounded assertion, and an R1/W1/C1 runtime-class taxonomy. The lineage’s one persistent gap never closed: classification without actuation – nothing in the constellation restarts, re-admits, or closes the loop.

Chapters 7–8 below are this project’s own answer to the gap the third lineage names.

1.4 Structure of this document

Chapters 2–6 instantiate `04_projection_and_scale.tex`: the measured comprehension–verification ratio at one instance scale (Chapter 2), the decidable-oracle discipline for mechanical trust (Chapter 3), the agent8 status byte and its branchless fleet-admission arithmetic (Chapter 4), the planetary-scale Fermi extrapolation of admission demand and supply, explicitly speculative (Chapter 5), the antibody clause bounding claim magnitude by receipting mechanism (Chapter 6), and – despite its position later in this document, among the chapters below – the retrofit applier and auto-nomic reconciliation tooling that apply this discipline to a running fleet (Chapter 10: every definition and proposition it states, including the worktree-isolation pair and the residual-vector/gap-report apparatus, is sourced from `04_projection_and_scale.tex`, not from `synthesis_thesis.tex`). Chapters 7–8 and 9, 11–13 instantiate `synthesis_thesis.tex`: supervision topology derived from the plan’s data-dependency DAG (Chapter 7), failure geometry with a completeness-by-construction

theorem (Chapter 8), the 10,000-member supervised-cell fault-injection measurements (Chapter 9), the Fable spec-driven-evaluation harness, evidence-bound in its mechanics and explicitly speculative in two flagged statements, a proposition and a remark (Chapter 11), the Merkle cell-rollup and write-ahead-log constructions (Chapter 12), and the named scope refusals that keep the whole document's own claims inside what it can receipt (Chapter 13). Chapter 14 closes.

Chapter 2

The Comprehension–Verification Gap, Instantiated at Scale

2.1 Measured and estimated quantities

The keystone’s Comprehension–Verification Gap theorem (Section 1.2 above, `thm:gap of projection.thesis.tex`) is asymptotic: comprehension cost grows without bound with the interior while boundary verification stays $O(\kappa)$. `04_projection_and_scale.tex` instantiates it on one concrete executed manufacture and, before quoting a ratio, disciplines which of its inputs are measured and which are estimated – a methodological convention this whole document follows.

Definition 2.1 (Instance quantities). For an executed manufacture σ : $T(\sigma)$, the interior token count (measured, a count of logged records); κ , the boundary field count of the receipt projection $\pi(\sigma)$ (a design constant); t_H , the per-frame recomputation wall-time (estimated from published hash throughput, not measured here).

2.2 The measured ratio, not an estimate

Both $T(\sigma)$ and κ in the theorem below are the measured/design-constant quantities of Definition 2.1, not estimates; the ratio that follows is therefore itself a measured consequence of the theorem’s asymptotics applied to one instance, not a new estimate layered on top of it.

Theorem 2.2 (Instance gap). *For comprehension cost $\text{Comp}(\sigma) = \Theta(T(\sigma))$ and boundary-verification cost $\text{Ver}_\partial(\sigma) = \Theta(\kappa)$, the instance ratio is $\Theta(T(\sigma)/\kappa) = \Theta(2.5 \times 10^6/4) \approx 6.25 \times 10^5$; both inputs are measured/design constants, so the ratio is a measured consequence, not an estimate.*

2.3 Boundary cost is invariant as the interior grows

The instance ratio of Theorem 2.2 would be uninteresting if it shrank as manufactures grew; the next proposition is what makes it a standing property of the frame schema rather than an artifact of one instance’s size.

Proposition 2.3 (Boundary invariance). *For manufactures $\{\sigma_i\}$ sharing the frame schema with $T(\sigma_i) \rightarrow \infty$, $\text{Ver}_\partial(\sigma_i) = \Theta(\kappa)$ is constant in i , and the estimated per-message wall-time $\kappa \cdot t_{cmp} + c \cdot t_H$ does not depend on $T(\sigma_i)$.*

2.4 Per-message wall time: an estimate, not a measurement

Speculative. The constancy of Proposition 2.3 is proved; the wall-clock seconds it takes is not measured in this document and is only a Fermi estimate from published BLAKE3 throughput [3].

Estimate 2.4 (Wall time). At BLAKE3 throughput 1–3 GB/s/core and a few-hundred-byte frame, $t_H \sim 10^{-7}$ s/frame; a four-field boundary compare is $\ll 10^{-7}$ s; per-message verification with $c \sim 10$ spot frames costs $\sim 10^{-6}$ s; these are order-of-magnitude estimates, the constancy-in- T point is the proved Proposition 2.3.

Chapter 3

Manufactured Trust: Decidable Oracles

3.1 Why admission needs a decidable oracle

Rice’s theorem specialized to observations (cited once, Section 1.2) rules out deciding whether any concrete implementation f meets its specification S by inspecting what f means. `04_projection_and_scale.tex` names the mechanical fallback: a decidable oracle, evaluated on a finite test set, that witnesses correctness without deciding it in general.

Definition 3.1 (Decidable oracle). For implementation $f : D \rightarrow R$ and specification $S \subseteq D \times R$, the correctness question is $Q_S(f) \equiv \forall x (x, f(x)) \in S$; a decidable oracle for f on finite $X \subseteq D$ is a total computable predicate $\Omega : X \rightarrow \{0, 1\}$ intended to witness $(x, f(x)) \in S$, terminating on every $x \in X$.

Proposition 3.2 (Necessity of the oracle move). *For S a non-trivial semantic property, $Q_S(f)$ is undecidable; hence no procedure decides $Q_S(f)$ for arbitrary f , and any mechanical trust in f must be trust in some Ω evaluated on finite X , plus an argument bounding residual risk on $D \setminus X$.*

The remainder of this chapter is that argument, instantiated three ways: differential agreement, admission-boundary fuzzing, and mutation testing.

3.2 Differential agreement between independent implementations

Definition 3.3 (Differential oracle). For $f, g : D \rightarrow R$ targeting the same S , the differential oracle is $\Omega_{f,g}(x) = [f(x) = g(x)]$; a test passes at x iff $\Omega_{f,g}(x) = 1$.

Proposition 3.4 (False-accept bound). *Under independent errors at rates p_f, p_g with wrong outputs drawn from $\geq M$ distinguishable values with collision probability $\leq 1/M$: $\Pr[\Omega_{f,g}(x) = 1 \wedge (x, f(x)) \notin S] \leq p_f p_g / M$; over a corpus of size n the probability some silent-agreement error survives all n tests is at most $n p_f p_g / M$, and a fixed silently-wrong input missed with per-input hit rate q has miss probability $(1 - q)^n \rightarrow 0$.*

Two independent implementations agreeing is weak evidence per test and strong evidence in aggregate; the bound is what turns “they agree” into a quantified residual-risk statement rather than a shrug.

3.3 Admission-boundary fuzzing

Definition 3.5 (Fuzzing oracle). For the total admission retraction $\alpha : \mathcal{O} \rightarrow \mathcal{O}^* \cup \{\perp\}$, the fuzzing oracle Ω_∂ maps generated $o \in \mathcal{O}$ to 1 iff $\alpha(o)$ terminates and yields either an admitted $x \in \mathcal{O}^*$ with well-formed artifact $\mu(x)$ or a refusal $\perp$ carrying a category in the eight-bucket taxonomy; $\Omega_\partial(o) = 0$ iff α crashes, diverges, or returns an unlabelled outcome.

Proposition 3.6 (Fuzzing soundness). *If α is total and terminating with codomain $\mathcal{O}^* \cup \{\perp\}$ and $\perp$ is refusal-with-category, then for every $o \in \mathcal{O}$, $\Omega_\partial(o) = 1$, and any observed $\Omega_\partial(o) = 0$ is a genuine defect in the retraction, not a property of the input.*

The soundness direction is what makes fuzzing worth running at all: a crash found by admission-boundary fuzzing is never dismissible as “the fuzzer generated a nonsense input” – α is total by construction, so nonsense inputs are exactly what it is required to refuse, not choke on.

3.4 The Mutant Kill Theorem

Mutation testing [4, 5] gives the third oracle. `04.projection_and_scale.tex` formalizes a validator as a staged pipeline and proves that, when every stage is sound and complete, the stage a mutant is caught at is not incidental – it is determined by which invariant the mutation broke.

Definition 3.7 (Staged validator). A staged validator is a pipeline $V = (V_1, \dots, V_k)$ where each stage $V_i : \mathcal{S} \rightarrow \{\text{pass}\} \cup \{\text{reject}_i\}$ tests a decidable invariant $I_i \subseteq \mathcal{S}$; the pipeline accepts s iff $s \in \bigcap_i I_i$, reporting on rejection the least i with $s \notin I_i$. Stage V_i is sound if $V_i(s) = \text{reject}_i \Rightarrow s \notin I_i$ and complete if $s \notin I_i \Rightarrow V_i(s) = \text{reject}_i$.

Definition 3.8 (Mutation, correct stage, kill). A mutation operator m maps a valid subject $s^* \in \bigcap_i I_i$ to a mutant $m(s^*)$ violating a non-empty set of invariants; its correct stage is $\mathbf{s}(m) = \min\{i : m(s^*) \notin I_i\}$; m is killed by V , written $\text{Kill}(m) = 1$, iff V rejects $m(s^*)$.

Theorem 3.9 (Mutant Kill Theorem). *For a staged validator $V = (V_1, \dots, V_k)$ in which every stage is sound and complete, and a mutation operator m with correct stage $\mathbf{s}(m)$: $\text{Kill}(m) = 1 \iff V_{\mathbf{s}(m)}$ rejects $m(s^*)$, and the least rejecting stage reported by V equals $\mathbf{s}(m)$.*

Corollary 3.10 (Diagnostic reading). *Under the Mutant Kill Theorem, the index of the killing stage certifies which law-bearing invariant a mutation broke; a non-empty surviving set that is not equivalent-mutants is a proof that the frame under-commits, the Faithful Projection converse.*

Corollary 3.10 is kernel-verified in both formalization lanes: `tools/paper-factory/lean-pilot/formalization` records `cor:diag` verified against kernel 4.31.0 in one attempt, and `tools/paper-factory/lean-lake/mathlib.mig` records it verified against Mathlib in two attempts with axioms `propext`, `Classical.choice`, `Quot.sound`. A surviving mutant that is not an equivalent mutant is therefore not merely a test-suite gap: by this corollary it is evidence the staged validator’s invariant set does not fully capture the law it is meant to enforce.

Chapter 4

The Agent8 Byte and Branchless Fleet Admission

4.1 The status byte as an OK-polarity projection

The eight-category refusal taxonomy that runs through this whole thesis series has an immediate machine-word encoding once it is projected to its OK polarity rather than its denial polarity.

Construction 4.1 (Agent8 byte). Project each agent a to a status byte $\mathbf{b}(a) \in \{0, 1\}^8$ whose lanes are the OK-polarity complements of the denial lanes (admitted, evidence, budget, authority, healthy, conformant, receipted, replayable); $\mathbf{1} = 0xFF$ is the all-OK byte; an agent is admissible iff $\mathbf{b}(a) = \mathbf{1}$, equivalently iff its 8-lane denial restriction is $\mathbf{0}$.

The corpus records this construction under the shared label `con:agent8` in both `04_projection_and_scale.tex`'s and `projection_thesis.tex`'s RDF – two distinct statements with one label, disambiguated in `docs/thesis/rdf/corpus.ttl` by `math:fromPaper`: this chapter's phrasing above, with the packed `b/0xFF` notation, is `04_projection_and_scale.tex`'s statement, while `projection_thesis.tex`'s sibling statement under the same label (“status byte $b \in \{0, 1\}^8$ whose lanes name admitted, evidence-ok, within-budget, authority-bound, healthy, conformant, receipted, replayable”) is the *same OK polarity* stated without the packed-byte notation – its lane names are the OK conditions themselves (“evidence-ok”, “healthy”), not their denials – so the two `con:agent8` statements are notational variants of one polarity, not an opposite-polarity pair. The two Lean files that back the `con:agent8` receipts disagree with *each other* in polarity, not with either source statement: `tools/paper-factory/lean-pilot/con.agent8.lean` formalizes the OK-polarity phrasing verbatim – its `byteAdmitted` is an `&&`-fold of eight named `Bool` fields (`admitted`, `evidenceOk`, ..., `replayable`) that is `true` exactly when every lane holds, i.e. the packed-byte Construction stated above with no `b/0xFF` numeral notation, so this file *is* an independent kernel check of the OK-polarity Construction. `tools/paper-factory/lean-lake/Praxis/Corpus/con.agent8.lean` instead reuses `con:denial`'s `Deny 8` type (a lane bit is `true` for a *denied* lane, $\perp$ the all-false clean/admitted byte, per that file's own `laneMap`) and defines `wordAdmits8 f` as the proposition `wordOf8 f = 0#64`: this is the denial-polarity dual of Corollary 4.4 below, not the OK-polarity phrasing shared by both source papers. Both polarities of `con:agent8` are therefore independently kernel-checked, one per Lean file. `tools/paper-factory/lean-pilot/formalization_receipts.jsonl` records `con:agent8` verified (kernel 4.31.0, one attempt); `tools/paper-factory/lean-lake/mathlib_migration.r` records it verified in the Mathlib lane in three attempts with no axioms used beyond the kernel's own.

4.2 Bit-plane layouts and the branchless admission word

The agent8 byte of Construction 4.1 can be laid out two ways in memory, and the layout choice is what determines whether fleet admission is a per-agent loop or a handful of word-parallel instructions.

Definition 4.2 (AoS/SoA layouts). The array-of-structures (AoS) layout stores each agent’s 8-bit byte contiguously, a 64-bit word holding 8 agents’ bytes; the structure-of-arrays (SoA / bit-plane) layout stores each lane ℓ as its own bitset $P_\ell \in \{0, 1\}^N$, a 64-bit word of P_ℓ holding the lane- ℓ bit of 64 agents.

Theorem 4.3 (Branchless admission). *Under the SoA layout, the admissibility of 64 agents is computed by the single branchless word expression $\text{adm} = P_1 \& \cdots \& P_8$; bit j of adm is 1 iff agent j has all eight lanes OK; the computation uses exactly 7 AND instructions, no data-dependent branch, and processes 64 agents, cost $7/64 \approx 0.11$ instructions per agent.*

4.3 The denial-polarity dual

Corollary 4.4 (One-denial dual). *In denial polarity, with $D_\ell = \neg P_\ell$ the 64-agent word of denial lane ℓ , the denied set is $\bigvee_\ell D_\ell$ and the admitted set its complement; fleet admission is the word-parallel join of denial lanes followed by one complement, and adding a ninth lane costs one more AND (resp. OR) per 64 agents.*

Theorem 4.3 and Corollary 4.4 are the same construction read in opposite polarity; extending the taxonomy beyond eight lanes – something the doctrine caps deliberately, per Chapter 8’s eight-member `FailureClass` – costs exactly one more instruction per 64 agents in either reading, not a redesign.

4.4 Borrow-safe SWAR zero-lane detection

Construction 4.5 (SWAR zero-lane mask). For a denial word $w \in \{0, 1\}^{64}$ of eight 8-bit lanes with $L_{\text{high}} = 0x8080808080808080$ and $L_{\text{low7}} = 0x7f7f7f7f7f7f7f7f$, the zero-lane bitmask is $z(w) = \neg(((w \& L_{\text{low7}}) + L_{\text{low7}}) \vee w) \& L_{\text{high}}$; bit 7 of lane j in $z(w)$ is 1 iff lane j in w is exactly `0x00`.

This is the SWAR (SIMD-within-a-register) idiom for finding zero bytes without a branch per lane and without leaking a borrow across lane boundaries; it is kernel-verified in both lanes (bare-core: one attempt; Mathlib: one attempt, no additional axioms), per the same two receipts files cited above under `con:swar`.

4.5 Popcount admission in one instruction

Theorem 4.6 (SWAR-verified sweep count). *For a 64-bit word w packing 8 agents gated against the broadcast required mask, the number of admitted agents is $\text{admitted}(w) = \text{popcount}(z(w))$, sweeping 8 agents in one CPU instruction with zero branches and no lane borrow leakage.*

Kernel-verified in both lanes: bare-core one attempt (kernel 4.31.0), Mathlib one attempt with axioms `propext`, `Classical.choice`, `Quot.sound`, per `tools/paper-factory/lean-pilot/formalization_receipts` and `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl`, label `thm:swar-verify`.

4.6 Memory footprint: an exact identity

Proposition 4.7 (Footprint identity). *A population of $N = 10^{10}$ agents, each an 8-bit byte, occupies $N \times 8 \text{ bits} = N \text{ bytes} = 10^{10} \text{ bytes} = 10 \text{ GB}$ in either layout; this is an exact arithmetic identity, not an estimate.*

4.7 Sweep time: an order-of-magnitude estimate

Speculative. The instruction count below is exact arithmetic given the constructions above; the wall-clock time it takes on any particular machine is not measured here and depends on memory bandwidth this document does not benchmark.

Estimate 4.8 (Sweep cost). A full admission sweep reads all 8 planes once (10 GB) and performs $7N/64 \approx 1.1 \times 10^9$ AND instructions for $N = 10^{10}$; at streaming bandwidth $B \sim 10^2\text{--}4 \times 10^2 \text{ GB/s}$, $t_{\text{sweep}} \approx 10 \text{ GB}/B \approx 25\text{--}100 \text{ ms}$; the instruction count is exact, the wall-time is an order-of-magnitude estimate.

Speculative, inheriting Estimate 4.8’s bandwidth assumption.

Corollary 4.9 (Throughput). *A single commodity server admits $N = 10^{10}$ agents in $\sim 25\text{--}100 \text{ ms}$, i.e. at $\sim 10^{11}\text{--}4 \times 10^{11}$ agent-admission decisions per second per server, bandwidth-bound.*

Chapter 5

Planetary Economics of Admission (Speculative)

This entire chapter is speculative. Every quantity in it is a Fermi estimate – an order-of-magnitude extrapolation to a planetary population of agents this document has not built, measured, or deployed. Nothing here is derived from the kernel-verified constructions of Chapter 4; it is motivated by them. No claim in this chapter should be read as evidence for anything outside it.

5.1 Demand

Speculation 5.1 (Admission demand). For a population $N \in [10^9, 10^{12}]$ agents each re-admitted at rate $\rho \in [1, 10^2] \text{ s}^{-1}$, aggregate admission demand is $\Lambda_{\text{demand}} = N\rho \in [10^9, 10^{14}]$ decisions/s, central estimate $\sim 10^{11}\text{--}10^{12}$ decisions/s for $N \sim 10^{10}$, $\rho \sim 10$.

5.2 Comprehension-based supply ceiling

Speculation 5.2 (Comprehension-supply ceiling). A comprehension-based admission decision costs $\sim 10^{-2}\text{--}10^0 \text{ s}$ of accelerator time, i.e. $\sim 1\text{--}10^2$ decisions/s per accelerator; a planetary fleet of $\sim 10^6\text{--}10^7$ accelerators yields a comprehension-supply ceiling $\Lambda_{\text{comp}} \lesssim 10^7\text{--}10^9$ decisions/s, central estimate $\sim 10^8$ decisions/s.

5.3 Bit-parallel supply

Speculation 5.3 (Bit-parallel supply). One commodity server supplies $\sim 10^{11}$ decisions/s bandwidth-bound; a rack of $\sim 10^2$ servers supplies $\sim 10^{13}$ decisions/s, $\sim 10^7\times$ cheaper per decision than an LLM judgment, so bit-parallel supply scales into the demand band on a single facility.

5.4 The affordability separation theorem

Speculation 5.4 (Affordability separation). For demand Λ_{demand} and supplies $\Lambda_{\text{comp}}, \Lambda_{\text{bit}}$: $\Lambda_{\text{comp}} \sim 10^8 \ll \Lambda_{\text{demand}} \sim 10^{11}\text{--}10^{12} \lesssim \Lambda_{\text{bit}} \sim 10^{11}\text{--}10^{13}$; comprehension-based admission is under-provisioned by three to four orders of magnitude for a 10^{10} -agent fleet, while bit-parallel admission meets demand on a single facility, and the separation persists under an order-of-magnitude perturbation of any single input.

Speculation 5.5 (Consequence). A planetary control plane that re-admits its population continuously cannot be built on comprehension: no attainable accelerator fleet closes the demand gap; it can be built on the sweep, whose cost is dominated by a single pass over 10 GB of memory; comprehension is reserved for sampled audit at honest $O(n)$ replay cost.

Despite being labelled “theorem” and “corollary” in the source material’s own environment names, both statements are Fermi-estimate consequences chained from three order-of-magnitude inputs (Estimates 5.1–5.3); this document keeps them in the SPECULATIVE register and environment throughout, naming the mismatch rather than inheriting it silently.

Chapter 6

The Antibody Clause: Claim Magnitude Bounded by Receipting

6.1 Claims and receipt witnesses

Chapter 5 was explicit that its numbers are not receipted. This chapter states the general mechanism that makes that distinction mechanical rather than a matter of authorial honesty: a claim's admissible magnitude is bounded by what its receipt witness can attest, full stop.

Definition 6.1 (Claim and witness). A claim is a pair $c = (\phi_c, w_c)$ where ϕ_c is a proposition and $w_c \in \mathbb{R}_{\geq 0}$ its asserted magnitude; a receipt witness for c is a receipt $r(c)$ whose boundary projection is accepted, $V(\pi(r(c))) = 1$, and whose committed fields entail an attested magnitude $\hat{w}_c$; a claim is admissible iff it has a receipt witness with $w_c \leq \hat{w}_c$.

Definition 6.2 (Overclaim set, antibody invariant). The overclaim set of a system emitting claims $\mathcal{C}$ is $\text{Over} = \{c \in \mathcal{C} : \nexists r(c) \text{ with } V(\pi(r(c))) = 1 \text{ and } w_c \leq \hat{w}_c\}$; the antibody invariant is $\text{Over} = \emptyset$, the docs-exceed-mechanism defect class enforced as a gate.

Note: docs-exceed-mechanism is the corpus's own name, quoted verbatim above from docs/thesis/rdf/04.pro. `def:over`, for the conceptual defect class this invariant guards against. It names no compiled gate, lint, or CI check in this repository: `grep -rn "docs-exceed-mechanism" crates/ src/` returns zero matches, and no `.rq` or `.sparql` file exists anywhere in the repository. The paper's own antibody-clause check, discussed below in this chapter, is likewise a proposed SPARQL query, not an implemented one.

6.2 The Magnitude Theorem

Theorem 6.3 (Magnitude). *If a system maintains $\text{Over} = \emptyset$, then for every emitted claim c , $w_c \leq \hat{w}_c = \max\{w : \text{the committed fields of } r(c) \text{ entail magnitude } w\}$, verifiable at boundary cost $O(\kappa)$ independent of the interior that produced c ; no claim can exceed what its mechanism can receipt.*

6.3 Reflexive application: this paper's own overclaim set

Corollary 6.4 (Reflexive application). *Each quantitative claim in this paper is either a theorem with a proof or an Estimate explicitly labelled with its inputs; the partition is exhaustive, so this*

paper’s overclaim set is empty, and by the magnitude theorem the magnitude of each claim is bounded by its witness as the invariant requires.

Note: this consolidated paper generalizes that theorem-or-Estimate partition one level, to the `doc:register` vocabulary (`packs/doc-pack/ontology.ttl`) that every paragraph in this file actually carries. Each quantitative claim here carries one of two `doc:register` values: `EVIDENCE-BOUND` – a theorem, proposition, or corollary with a proof, or a Measurement reporting values observed from an executed run (Chapter 9) – or `SPECULATIVE` – an Estimate, or a statement kept in the `speculative` environment and explicitly labelled with its inputs, including Fermi chains that the source material itself calls “theorem”/“corollary” but that this document keeps under the speculative register (Chapter 5’s naming mismatch is disclosed where it occurs rather than inherited silently). This register partition is exhaustive over the `theorem`, `proposition`, `corollary`, `estimate`, `speculative`, and `measurement` environments actually used in this document, so Corollary 6.4’s partition still holds under this finer-grained labeling, and this paper’s overclaim set remains empty under either partition. This document’s own `doc:register` property (evidence-bound / speculative on every paragraph, per `packs/doc-pack/ontology.ttl`) would be the concrete, checkable substrate for a SPARQL query over `rdf/thesis.ttl` that finds any paragraph with neither register value, or any speculative paragraph in a chapter titled without the word “speculative” next to an evidence-bound one – exactly a check for $\text{Over} \neq \emptyset$ in the antibody-clause sense above. No such query is implemented in this repository today (Chapter 6’s opening section notes the same gap for `docs-exceed-mechanism` generally); this paragraph describes what the check would be, not a check that runs.

Chapter 7

Derived Supervision Topology

7.1 Stage depth from the data-dependency DAG

`synthesis_thesis.tex` derives supervision structure rather than accepting it as hand-authored, starting from the plan’s own data-dependency edges – the same DAG that `crates/praxis-synthesis/src/dag.rs` builds and that the solver’s saturated fact base already commits to before any node runs.

Definition 7.1 (Stage depth). Let $G = (V, E)$ be the plan’s data-dependency DAG (edge $u \rightarrow v$ iff an add-effect of u feeds a precondition of v); define $d(v) = 0$ for sources and $d(v) = 1 + \max_{u \rightarrow v} d(u)$ otherwise. A stage $S_k = \{v : d(v) = k\}$.

Kernel-verified in both lanes (bare-core and Mathlib, label `def:depth`), per `tools/paper-factory/lean-pilot/` and `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl`.

7.2 Restart strategy earned from the dependent set

Every stage in Definition 7.1 needs a restart policy when one of its members crashes, and `synthesis_thesis.tex` derives which policy from the same dependency edges rather than letting an author pick one – implemented in `crates/praxis-synthesis/src/supervise.rs`.

Definition 7.2 (Earned restart strategy). Stage S_k supervises `RESTARTFORONE` iff some $v \in S_k$ has a nonempty transitive dependent set $D(v)$; otherwise `ONEFORONE`. The cohort of v is $\{v\} \cup D(v)$ under `RESTARTFORONE` and $\{v\}$ under `ONEFORONE`.

7.3 The sealed derive constructor and topology_hash

Proposition 7.3 (Sealed topology). $\mathcal{T} = (\text{stages}, \text{policy})$ is produced only by `derive` (a sealed constructor: no literal construction compiles), and `topology_hash = ca(stages || policy || plan_hash || problem_hash)` joins the receipt lineage. Determinism is test-pinned: same plan, same hash.

There is no code path by which a human-authored supervision tree can enter the system: the only constructor that yields a `SupervisionTopology` value is `derive`, and its output is folded into the receipt chain by `topology_hash` before any node executes.

7.4 Refusal: no OneForAll

Refusal 7.4 (No OneForAll). No ONEFORALL: OTP justifies one-for-all by shared mutable fate between siblings. In an acyclic data-flow plan, same-depth siblings are independent by construction – the coupling one-for-all encodes is inexpressible. The variant does not appear in the Strategy enum at all; an exhaustive-match test makes its future addition a compilation event, forcing the doctrine conversation.

7.5 Refusal: no SimpleOneForOne, no clamped intensity

Refusal 7.5 (No SimpleOneForOne, no clamping). No SIMPLEONEFORONE; intensity > 8 refused, not clamped: derived plans have static step sets, and `RestartPolicy::new(9, _)` returns a Refusal – the byte governor applies to recovery itself, and silently clamping a stated policy would be the executor editing the operator’s law.

Kernel-verified: bare-core, verified in one attempt (kernel 4.31.0); Mathlib lane, verified in two attempts with axioms `RestartPolicy`, `no_simple_one_for_one`, `RestartPolicy.new` – per the same two receipts files, label `refusal:simpleoneforone`.

Chapter 8

Derived Failure Geometry

8.1 The eight-member FailureClass enum

A supervision topology says who restarts with whom; it does not say what a given failure was. `synthesis_thesis.tex` closes that gap with a closed, eight-member failure taxonomy – implemented in `crates/praxis-synthesis/src/geometry.rs` – capped at eight by the same doctrine that bounds the `agent8` taxonomy of Chapter 4.

Definition 8.1 (FailureClass). `FailureClass = {LogicFault, BudgetBreach, AuthorityVacuum, TransientFault, Stall, StarvedInput, CertifiedUnsat, GeometryGap}` – the semantic axis, capped at eight by the doctrine.

Kernel-verified in both lanes: bare-core verified; Mathlib verified with axioms `propext`, `Classical.choice`, `Quot.sound`, label `def:classes`.

8.2 Branches and first-match classification

Definition 8.2 (Branch, geometry). A branch is (c, Σ, r) : a class, a conjunction of signal predicates over a crash snapshot, and a lawful response $r \in \{\text{Restart}, \text{Park}(\rho), \text{Refuse}(\text{core}), \text{Escalate}\}$. The geometry Γ maps each node to an ordered branch list; classification is first-match-wins; `geometry_hash = ca( $\Gamma \parallel$  topology_hash)`.

Chaining `geometry_hash` to `topology_hash` (Proposition 7.3) means a failure geometry cannot be attached to a plan after the fact without changing the receipt of the plan it claims to cover.

8.3 Completeness by construction: the gap outcome

The eighth member of Definition 8.1, `GeometryGap`, is not an ordinary failure class: it is the system’s own admission that its geometry can be incomplete.

Theorem 8.3 (Geometry-gap completeness). *GeometryGap is the value classification returns when no stored branch matches; it is never a member of any stored list. Consequently no branch ordering, addition, or deletion can capture, mask, or remove the gap outcome: the map admits its own incompleteness by construction, not by convention.*

This is the same guarantee, at the failure-classification layer, that `Rfsl` gives at the admission layer in the keystone equation: an outcome that names “I don’t have a rule for this” rather than

silently defaulting. The corpus records this theorem under the shared label `thm:gap` in both `synthesis_thesis.tex`'s and `projection_thesis.tex`'s RDF – two distinct statements with one label, disambiguated in `docs/thesis/rdf/corpus.ttl` by `math:fromPaper`; to avoid that collision this document uses its own label, `thm:geomgap`, for the statement above.

8.4 Fragile preconditions

Not every recoverable-looking fact is actually recoverable; the geometry needs a name for the ones that are not.

Definition 8.4 (Fragile precondition). A precondition predicate p of capability c is fragile iff no capability in the problem produces p . The initial state is a one-time gift: if a fragile fact is lost mid-run, nothing in the plan can lawfully re-produce it – whereas a fact with even one producer is recoverable by restarting that producer.

8.5 Total accounting

Proposition 8.5 (Total accounting). *Every node of every supervised run carries exactly one disposition (`Completed(r)`, `Parked`, `SkippedBy`, `GaveUp`); no silent rows exist (test-pinned: $|\text{dispositions}| = |V|$).*

Together, Theorem 8.3 and Proposition 8.5 are the failure-geometry analogue of the antibody clause (Chapter 6): every node has a named outcome, and any outcome the geometry cannot name surfaces as `GeometryGap` rather than disappearing.

Chapter 9

Supervised Execution: Cell Measurements

9.1 Recovery is invisible in the receipt chain

Chapters 7–8 derive structure before any node runs; this chapter is the paper’s own primary empirical contribution, reporting what happens when that structure is exercised by actually-injected faults.

Measurement 9.1 (Recovery is invisible). On the lawobject plan with two injected transient crashes at judge: both crashes land in named branches (TransientFault, then Stall), the run completes, and the final root hash is byte-identical to the crash-free run’s. Park-then-re-admit heals to the same identity; machine death composes: kill-9 mid-restart-loop, WAL recovery, identical receipt.

Kernel-verified in both lanes (label `meas:recoveryinvisible`), bare-core in one attempt, Mathlib in one attempt with axiom `meas_recoveryinvisible.recoveryinvisible`.

9.2 The 10,000-member cell

Measurement 9.2 (10,000-member cell). The supervised cell, 10,000 members, release: overhead of supervision at fault rate 0 is -0.9% of medians over five paired runs, inside the baseline’s own $\pm 5.7\%$ run spread – statistical noise. Recovery counts track injected transient rates exactly: 69/608/3,179 recovered at 1%/10%/50%. Throughput is flat across all fault rates. The crash-looping template is detected at quorum and quarantined by epoch 2 in every configuration.

Kernel-verified in both lanes, label `meas:cell`: bare-core one attempt, Mathlib two attempts with axiom `CellMeasurement.overhead_within_baseline_spread` (recorded as not depending on any further axioms). Implemented in `crates/praxis-synthesis/src/cell_supervise.rs` and `crates/praxis-synthesis/src/fault.rs`.

9.3 What flat throughput means

Remark 9.3 (Flat throughput). What flat throughput means: the cost of a supervised fleet tracked its failure novelty, not its failure rate: recovered transients are absorbed at boundary cost, and the one genuinely novel pathology (the crash-looping class) was converted — once, by quorum — into a cheap standing refusal. The full novelty-curve-under-faults re-measurement is receipted as deferred, not claimed.

9.4 The deferred novelty-curve measurement

The remark above names a measurement this paper does not have; the following refusal is what keeps that admission from being a silent omission.

Refusal 9.4 (No novelty-curve measurement yet). No novelty-curve-under-faults measurement – yet: member-level fault injection recovers without re-running solver work, so a work-proxy re-measurement would overstate the cache dividend – the flattering error. The claim is withheld until node-level injection is wired through the fleet path; the receipt file names this in its own deferred list.

This refusal is itself kernel-verified (label `ref:curve`): bare-core two attempts; Mathlib one attempt, with axioms `Praxis.Corpus.RefCurve.NoveltyCurveUnderFaultsWithheld` and `Praxis.Corpus.RefCurve` explicitly documented as project-specific epistemic markers with no Mathlib equivalent – a formalization of “we know we have not measured this,” not of the measurement itself.

Chapter 10

Fleet Application Tooling: Retrofit and Reconciliation

The oracle discipline of Chapter 3 and the supervised-execution discipline of Chapters 7–9 both need a way to apply a change to a running fleet without ever letting a failing change reach the fleet’s shared branch, and a way to measure what to change in the first place. That pair of mechanisms – an AST-gated, Git-worktree-isolated applier (worktree creation, the change commit, and the push back to origin implemented in `crates/praxis-retrofit/src/fleet_apply.rs`; the CI-gate battery and the unconditional rollback-on-failure protocol implemented in the sibling file `crates/praxis-retrofit/src/fleet_validate.rs`) and the residual-vector/gap-report reconciliation apparatus that names the applier’s target – is this chapter’s subject, matching the two halves of its own title, retrofit and reconciliation; it is fleet-application tooling, not an oracle-trust argument, and is placed here rather than beside Chapter 3 for that reason. Two disclosures on how `fleet_validate.rs` relates to the definitions below, checked directly against that file: first, its `capture_git_state/restore_git_state` functions and `GitState` struct back the isolation-and-unconditional-rollback mechanism of Definition 10.3, Definition 10.4, and Proposition 10.5 – the struct’s field is literally named `commit_sha`, which is why the three definitions below render the rollback target as `commit_sha` rather than the corpus’s `baseline_sha` token (`docs/thesis/rdf/04_projection_and_scale.ttl`’s `def:worktree`, `def:worktree-ret`, and `prop:mainclean` all use `baseline_sha`; this is the one token substitution made against the verbatim-quotation rule of Chapter 1, disclosed here rather than three times). Second, the file’s `CiGateName` enum does *not* back the three-gate CI oracle battery (`cargo build`, `cargo test`, `cargo clippy`) that Definition 10.4 quotes from the corpus: the actual enum has five variants (`Fmt`, `Clippy`, `Test`, `Deny`, `Typos`), running `cargo fmt --check`, `cargo clippy`, `cargo test`, `cargo deny check`, and a `typos` check respectively, with no build gate at all. Definition 10.4’s gate list is quoted verbatim from the corpus and is left unchanged below; it names the CI battery as the `04_projection_and_scale.tex` corpus statement described it at the time, not as `fleet_validate.rs`’s `CiGateName` enum stands today. Every labeled definition and proposition below is, in the source-attribution sense of Chapter 1, drawn from `04_projection_and_scale.tex` rather than `synthesis_thesis.tex`, even though this chapter sits among the chapters that otherwise instantiate the latter: `def:astgate`, `prop:astterm`, `def:worktree`, `def:worktree-ret`, `prop:mainclean`, `def:residual`, `def:vizgap`, and `prop:vizgap` are all `04_projection_and_scale.tex` statements per `docs/thesis/rdf/corpus.ttl`’s `math:fromPaper`; this chapter is placed here because its subject (fleet application tooling) follows naturally after the supervised-execution material, not because its source paper does.

10.1 AST gates and the isolated worktree applier

Definition 10.1 (AST gate). An AST gate is a syntactic predicate $g : \mathcal{T} \rightarrow \{0, 1\}$ on an abstract syntax tree $\mathcal{T}$, computable in $O(|\mathcal{T}|)$ time; the retrofit applier gates every proposed change through an ordered battery $(g_1, \dots, g_m)$: the change is admitted iff all gates pass, $\bigwedge_i g_i(\mathcal{T}) = 1$; a failing gate returns a denial with the gate index and offending AST node.

Proposition 10.2 (Battery termination). *For any finite AST $\mathcal{T}$ and finite gate battery $(g_1, \dots, g_m)$ of syntactic predicates, battery evaluation terminates in $O(m \cdot |\mathcal{T}|)$ time and returns either **Admitted** or a refusal carrying the first failing gate index and offending node; the retraction is total, never diverging and never returning an unlabelled outcome.*

10.2 Git-worktree isolation end to end

The gate battery of Definition 10.1 decides admission of a change; isolating the attempt to apply it from the fleet’s shared branch is a second, orthogonal mechanism.

Definition 10.3 (Worktree isolation). An applier isolates modifications by spawning a decoupled working tree via `git worktree add temp_path phase_branch`, running validation gates and executing the auto-rollback protocol `git reset --hard commit_sha` on failure to guarantee main branch cleanliness.

Definition 10.4 (Worktree-isolated CI application). The retrofit applier isolates each application attempt in a fresh Git worktree, applies the change, runs the CI oracle $(g_1, \dots, g_m)$ (`cargo build`, `cargo test`, `cargo clippy`), and on any gate failure executes `git reset --hard commit_sha`; on all-pass the worktree change is merged and the chain receipt for the application is minted with the worktree OID committed into the frame’s `obj_refs` payload digest.

Proposition 10.5 (Main stays clean). *Under the isolated worktree applier, the main branch is never advanced past a failing CI gate; the rollback protocol is unconditional on failure, running before the worktree is removed, restoring the branch to `commit_sha` regardless of which gate failed; hence the main branch satisfies all CI gates at every point in its history.*

Definitions 10.3 and 10.4 name the same isolation mechanism at two levels of detail – the first as the coarse-grained mechanism statement, the second the more operational, CI-oracle-specific form – and both are `04_projection_and_scale.tex` statements (per `docs/thesis/rdf/corpus.ttl`’s `math:fromPaper`; `synthesis_thesis.tex` has no worktree material at all). Proposition 10.5 is the guarantee both are stated to establish: the operator’s global `/Users/sac/CLAUDE.md` fix-forward-only invariant (no destructive git operations on the shared history) – distinct from this repository’s own `CLAUDE.md`, which carries a different, narrower set of invariants (Chapter 13 below cites one) – is exactly what an unconditional, gate-failure-triggered rollback of an isolated worktree is built to enforce mechanically.

10.3 Autonomic reconciliation: residual vectors and gap reports

A gated, isolated applier still needs to know what to change; autonomic reconciliation is the measurement step that produces that target, and it obeys the same $O(\kappa)$ boundary-cost discipline as the receipt projection of Chapter 2.

Definition 10.6 (Residual vector). For environmental observations O and target artifact A , the measurement $\mu(O)$ returns a residual vector $R \in \mathbb{R}^k$ with $r_i = \text{measured}_i - \text{midpoint}(\text{target}_i)$; reconciliation selects the dominant dimension $\arg \max_i |r_i|$ and applies a repair operator subject to **RepairBand** limits.

Note: **RepairBand** is the corpus’s own name for this limit, quoted verbatim above; it is not a named type in this codebase today. The sketch at `crates/praxis-reconciler/src/loop_logic.rs` is not a Cargo workspace member (absent from the `[workspace] members` list in `Cargo.toml`) and imports its operator/report types from a `genesis_types_v2` crate that does not exist anywhere in this repository, so no **RepairBand** or other bound-enforcing type is compiled or checkable today; the definition above states the design intent, not a verified property of running code.

Definition 10.7 (Visual gap report). A visual gap report is the output of `measure_gap`: for each reconcilable dimension $i \in \{1, \dots, k\}$, the residual $r_i = \text{measured}_i - \text{midpoint}(\text{target}_i)$ and the dominant dimension $i^* = \arg \max_i |r_i|$ together with a rendered diff block; the report has k residual values and one dominant index, size $O(k)$ independent of the interior.

Proposition 10.8 (Gap report cost). *The visual gap report gives $\text{gap} : \mathbb{R}^k \rightarrow \mathbb{R}^k \times [k]$, a projection with the same cost structure as the receipt projection, $O(k)$ to compute and $O(k)$ to verify; the repair operator acts on the dominant dimension i^* subject to **RepairBand** bounds, producing at most one corrective actuation, a bounded deterministic manufacture.*

Note: as Definition 10.6’s note above discloses, **RepairBand** is not yet a compiled, named bound type in this repository, so the bound above is the corpus’s stated design intent, not a verified property of running code. Reconciliation therefore never inspects more than $O(k)$ reconcilable dimensions to decide its one corrective action, the same shape of argument Chapter 2 makes for receipt verification at $O(\kappa)$: a bounded-size projection stands in for an unbounded interior.

Chapter 11

Fable Harness: Spec-Driven LLM Evaluation

11.1 Harness mechanics

The oracle discipline of Chapter 3 applies directly to evaluating whether a language-model-generated implementation is correct; `synthesis_thesis.tex`’s Fable harness (`crates/rust-fable-testbed`, `docs/fable/`) is that oracle instantiated for LLM-generated code, run hermetically against a mock model membrane so the verdict never depends on a live API call.

Definition 11.1 (Fable harness). A Fable harness is a test environment that constructs a prompt P_T from a task ontology T , calls a mock model membrane M that intercepts the LLM API and returns a deterministic response drawn from a test fixture indexed by P_T ’s hash, extracts the code block C from the model response R , and runs the verification oracle $\text{Oracle}(T, R)$ against C in an isolated cargo workspace. The mock membrane ensures that harness runs are hermetic: no real LLM call is made, the response is deterministic given P_T , and the oracle verdict is reproducible.

Definition 11.2 (Sandbox oracle). Let P_T be a prompt compiled from task ontology T , and let C be the code block extracted from model response R . The sandbox verification oracle is the composite: $\text{Oracle}(T, R) = \text{cargo_build}(C) \wedge \text{cargo_test}(C) \wedge \text{cargo_clippy}(C) \wedge \text{safety_audit}(C)$. The outcome is receipted and chained to the ledger using rolling BLAKE3 hashes [3].

Both are kernel-verified in both lanes (`def:fable`, `def:sandbox`); the Mathlib-lane receipt for `def:sandbox` names five project-specific axioms (`Praxis.Corporis.DefSandbox.{blake3Chain,cargoBuild,cargoO` rather than proving the four external tools’ behavior from first principles – the formalization is of the composite’s shape, not a re-verification of `cargo` itself.

11.2 Receipting fable runs

Construction 11.3 (Fable receipt chain). When $\text{Oracle}(T, R) = 1$, the harness mints a receipt using the standard `OcelCausalFrame` chain: $h_+ = H(h_- \parallel \text{body}(\text{fr}))$, where the frame’s `obj_refs` carry $\text{dg}(C) = H(\text{canonical bytes of } C)$. The genesis seed is $H(\text{“fable-v1-genesis”})$, domain-isolated from the plan chain so fable receipts cannot cross-verify with plan receipts even if their terminal hashes collide.

Kernel-verified in both lanes, label `con:fablechain`: bare-core one attempt; Mathlib two attempts with axioms `chainH`, `propext`.

11.3 Speculative: decidability and differential-oracle claims

Speculative, and flagged as under-supported. The harness mechanics above (Definitions 11.1, 11.2, Construction 11.3) are evidence-bound: they describe what the harness does and how it receipts. Whether $\text{Oracle}(T, R)$ is a decidable oracle in the sense of Definition 3.1 is a separate, stronger claim, and the corpus RDF for `synthesis_thesis.tex` itself marks both proof steps of that claim UNRESOLVED: in `synthesis_thesis.tex`’s own, standalone RDF they lean on an external paper’s (“Paper 04”) Definition 4.7 and Proposition 4.4. Unlike in that standalone source, those two statements are not external to *this* document: Paper 04 is `04_projection_and_scale.tex`, which this consolidation folds in as Chapters 2–6 and 10. `synthesis_thesis.tex`’s own “Definition 4.7” / “Proposition 4.4” numbering does not resolve against this paper’s actual chapter/theorem counters (`04_projection_and_scale.tex`’s own Chapter 4 has one definition and a Proposition 4.4 of its own, Proposition 4.7, neither of which is the statement being cited), so the two citations are matched here by *content*, not by number: the staged validator that step cites is Definition 3.7, and the “genuine defect, not a property of the input” soundness result that step cites is Proposition 3.6, both of Chapter 3 above, both already stated and kernel-checked in this corpus. That resolves the *availability* gap but not the *proof* gap: Proposition 3.6’s “genuine defect, not a property of the input” argument is proved for the admission retraction α , and the Mutant Kill Theorem (Theorem 3.9) needs every stage of a staged validator to be individually sound *and* complete (Definition 3.7) to license the analogous claim for $\text{Oracle}(T, R)$ ’s four stages (`cargo_build`, `cargo_test`, `cargo_clippy`, `safety_audit`); no statement in this corpus proves that those four specific stages are each sound and complete decidable invariants in Definition 3.7’s sense. This document therefore narrows, rather than resolves, the earlier framing’s gap: the cited results are now in-repo and in-corpus, but instantiating them to $\text{Oracle}(T, R)$ still requires a soundness/completeness argument for the four sandbox stages that has not been made.

Speculation 11.4 (Fable oracle decidability). $\text{Oracle}(T, R)$ is a decidable oracle for the correctness question “does C implement T correctly?” on the finite test corpus generated by the harness: it is total, terminating, and labelling (pass or fail with a structured error from the failing stage).

Speculation 11.5 (Mock membranes as differential oracles). Mock membranes as differential oracles: the mock membrane produces responses from test fixtures; a real-model run produces a different response. Running both and comparing (differential oracle) bounds the false-accept rate: an implementation error that passes the oracle with the mock response but fails with the real response is caught by the differential.

Both statements are kernel-admitted (label `prop:fable-oracle`: bare-core and Mathlib, two attempts each, no additional axioms in the Mathlib lane; label `remark:mockmembranes`: bare-core two attempts, Mathlib one attempt with axiom `propext`) – but kernel admission here certifies that the statement was formalized as declared, not that its unresolved dependency on the external Paper 4.7/4.4 result has been discharged. Until that dependency is either folded into this corpus or independently re-derived, Proposition 11.4 and Remark 11.5 stay in this speculative section, not beside Section 11.1’s evidence-bound harness mechanics.

Chapter 12

Hierarchical Merkle Cell Rollups and Write-Ahead Logs

12.1 Cell rollup

The 10,000-member cell of Chapter 9 needs a receipt shape that scales to that membership count without linearizing every member’s hash into one flat chain; a two-level Merkle rollup, implemented in `crates/praxis-synthesis/src/cell.rs` and `crates/praxis-synthesis/src/cell_supervise.rs`, is that shape.

Construction 12.1 (Merkle cell rollup). Let $M_{g,i}$ be the terminal hash of member i in group g . The Group Receipt hash is $H_g = H(\text{sort}(\{M_{g,i}\}))$, and the Cell Receipt hash is the rolling fold over G group hashes: $H_{\text{cell}} = H(H_1 \parallel H_2 \parallel \dots \parallel H_G)$.

Kernel-verified in both lanes, label `con:merklecell`: bare-core three attempts; Mathlib two attempts with axiom `chainH`.

12.2 Write-ahead log frames

Memoized DAG nodes (Chapter 7) must survive a crash mid-write without corrupting the cache they journal into; a length-prefixed, digest-committed frame, implemented in `crates/praxis-synthesis/src/wal.rs`, is what makes a torn write detectable rather than silently accepted on recovery.

Definition 12.2 (WAL frame). A cache frame journaling the memoized DAG nodes is serialized as a length-prefixed frame: $\text{WAL_Frame} = \langle \text{length}, H(\text{payload}), \text{payload} \rangle$, allowing recovery to discard torn frames.

Kernel-verified in both lanes, label `def:walframe`: bare-core two attempts; Mathlib one attempt with axiom `chainH`.

Chapter 13

Scope Discipline: The Refusal Register

The antibody clause of Chapter 6 bounds what a claim may assert; this chapter is its dual, naming what this project explicitly declines to claim or adopt, so that the omission is a receipted decision rather than a silent gap.

13.1 No path dependency on knhk

Refusal 13.1 (No knhk path dependency). No path dependencies on knhk: verified by live probes, recorded as the standing reason – the leanest candidate crate drags tokio/reqwest/opentelemetry transitively; the mu-kernel ships property-testing libraries as regular dependencies and uses unsafe transmutes; the workspace does not build as a whole. Ports of 50–200-line designs, tagged PORT(knhk) with per-item deltas, cost less than the supply chain. Drift is greppable.

Kernel-verified in both lanes, label `refusal:noknhkpath`: bare-core one attempt; Mathlib two attempts with axioms `knhkTransitiveDeps`, `knhk_drags_disallowed_deps`, `propext`, `Classical.choice`, `Quot.sound`. This is the same discipline named in this repository’s own CLAUDE.md: “praxis-synthesis deps frozen to exactly: pddl-index, chatman-common, blake3, serde, serde_json, thiserror,” enforced by `tests/no_llm_runtime.rs`.

13.2 No measured CPU ticks

Refusal 13.2 (No fabricated tick measurement). No measured CPU ticks: knhk’s own hot receipts carried dummy tick values behind a hardcoded 4 GHz assumption. Declared deterministic costs (Ticks as an abstract unit) are the honest model for a planner; rdtsc instrumentation is a bench follow-up, documented as a divergence, not smuggled in as a claim.

Kernel-verified in both lanes, label `refusal:nomeasuredticks`: bare-core one attempt; Mathlib two attempts with axioms `declaredTicks`, `rdtscTicks`. Read together with Section 2.4’s speculative wall-time estimate, the contrast is the point: an estimate labelled speculative and an unmeasured quantity smuggled in as fact are not the same defect, and this refusal is what keeps this project on the labelled-estimate side of that line for its own tick accounting.

13.3 v1 scope, named

Refusal 13.3 (v1 scope). v1 scope: beat scheduling, full R1/W1/C1 SLO matrix, quarantine parole, and supra-cell supervision. Each receipted with salvage: the enum and one tier transition ship;

epoch boundaries already provide the re-admission cadence beats would refine; quarantined classes await a parole mechanism; cells supervising cells is the composition step after this one.

Kernel-verified in both lanes, label `refusal:v1scope:` bare-core one attempt; Mathlib one attempt, no axioms beyond the kernel's own. This is the same v1-scope discipline the milestone ticket dependency graph applies at the project level ([docs/jira/v26.7.4/tickets/index.md](https://docs.jira/v26.7.4/tickets/index.md)): a named boundary with a stated next step, not an unstated one.

Chapter 14

Conclusion

This paper showed two halves of one fleet lifecycle. The admission half (Chapters 2–6) instantiates the keystone’s Comprehension–Verification Gap at one measured scale, gives a three-part decidable-oracle discipline for trusting concrete implementations, constructs a branchless byte-level admission sweep with an exact memory-footprint identity, and states the antibody clause that bounds every claim – including this paper’s own – by what its receipt witness attests. The supervision half (Chapters 7–13) derives, rather than hand-authors, the stage structure and failure geometry that protect a running fleet, reports a 10,000-member cell’s actual fault-injection numbers rather than an extrapolation of them, extends the same gate-and-isolate discipline to a live retrofit applier and to a spec-driven LLM evaluation harness, and names – rather than silently carries – both its unresolved dependency (the Fable oracle’s decidability argument) and its declined-scope items. Nothing in this document’s speculative chapters and sections (Chapter 5; Section 11.3) is mixed with its evidence-bound claims without a visible marker; per Corollary 6.4 and the reflexive reading of Chapter 6, that partition is this document’s own overclaim set, and it is empty by construction, checkable by the same SPARQL query that checks any other paper in this series.

Bibliography

- [1] The Praxis / Capability Physics Program. The Projection Principle: A First-Principles Theory of Bounded Receipted Execution for a Trillion Agents Beyond One Another’s Comprehension. Keystone thesis of this series, 2026.
- [2] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 2003 (reprint of the 1953 original), pp. 5–51.
- [3] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. *BLAKE3*: One function, fast everywhere. Preprint, 2020.
- [4] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4), 1978, pp. 34–41.
- [5] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5), 2011, pp. 649–678.